

Reviewer Pack — Sprague-Grundy Game Value of CNF Bounds Tree-Frege Lines

Entry 501b089a21b9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 14:25:26 UTC

Sprague-Grundy Game Value of CNF Bounds Tree-Frege Lines

Entry ID: 501b089a21b9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 14:25:26 UTC

1. Conjecture

Field A (mathematical branch): Sprague-Grundy / Nim-value theory of impartial combinatorial games (Bouton 1901; Sprague 1935; Grundy 1939; Berlekamp-Conway-Guy ‘Winning Ways’ tradition); a branch of combinatorial game theory rarely deployed in proof complexity, distinct from Karchmer-Wigderson games and from Ehrenfeucht-Fraïssé pebble games used in descriptive complexity. **Field B** (complexity object): Tree-like Frege refutation line count $L_{TF}(F)$ on small unsatisfiable 3-CNFs, accessed via Reckhow’s polynomial equivalence with tree-resolution leaf count $L_T(F \vdash \perp)$ (depth-0 Frege over CNFs), serving as a tractable proxy for the Krajíček-Buss-Razborov-Impagliazzo EF lower-bound program.

Statement:

Define the impartial Variable-Splitting Game $V(F)$ on a CNF F as follows: a state is a CNF obtained from F by a sequence of literal substitutions with clause simplification (drop satisfied clauses, drop falsified literals from surviving clauses); a move from state F' picks any variable v occurring in F' together with a value $b \in \{0,1\}$ and transitions to $F'[v=b]$; terminal states (no remaining variables) carry Sprague-Grundy value 0, and non-terminal values are the mex of children’s values. Let $G(F)$ denote this Grundy value. Conjecture: for every unsatisfiable 3-CNF F with $n \leq 20$ variables, $\lceil \log_2(1+G(F)) \rceil \leq \lceil \log_2 L_T(F \vdash \perp) \rceil$, where L_T is the leaf count of lex-DPLL with unit propagation; a single unsatisfiable 3-CNF violating this inequality refutes the conjecture.

Rationale (proposer’s reasoning):

Sprague-Grundy values capture an exponential structural complexity intrinsic to the move-graph of CNF splitting, yet have never been linked to proof-system line counts; if the inequality holds, $G(F)$ yields a Frege-style certificate of refutation hardness computable without exhibiting any refutation, opening a non-Boolean-Fourier route from impartial-game algebra to EF size lower bounds via Reckhow’s reduction. The Grundy value is invariant under literal-flipping and variable-renaming, matching a symmetry class compatible with natural-proofs-safe invariants. Failure modes (small $G(F)$ with large L_T) are equally informative, exposing a genuine gap between game-theoretic disjunctive complexity and proof-theoretic refutation length.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a809f37c14bec8be

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~ 25 unsat 3-CNF instances per (n, α) cell with $n \in \{6, 8, 10, 12\}$ and $\alpha \in \{4.5, 5.0, 5.5\}$ (~ 300 instances), conjecture is SUPPORTED iff every instance satisfies $\lfloor \log_2(1+G(F)) \rfloor \leq \lfloor \log_2 L_T(F \vdash \perp) \rfloor$ AND Pearson $r(\log_2(1+G), \log_2 L_T) \geq 0.6$ pooled.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Sprague-Grundy impartial game CNF satisfiability Grundy value - tree resolution DPLL leaf count combinatorial game Grundy - variable splitting game proof complexity tree-like Frege Nim-value

Top relevant hits considered: - [<http://arxiv.org/abs/1912.12573v3>] A base-p Sprague-Grundy type theorem for p-calm subtraction games: Welter's game and representations of generalized symm - [<http://arxiv.org/abs/1702.07068v1>] The Sprague-Grundy function for some nearly disjunctive sums of Nim and Silver Dollar games - [<http://arxiv.org/abs/1507.04665v2>] Invariance of the Sprague-Grundy Function for Variants of Wythoff's Game - [<http://arxiv.org/abs/2207.14140v1>] Playing a 2D Game Indefinitely using NEAT and Reinforcement Learning - [<http://arxiv.org/abs/1104.3098v3>] Optimal strategies for a game on amenable semigroups - [<http://arxiv.org/abs/math/0501315v2>] Taming the wild in impartial combinatorial games - [<http://arxiv.org/abs/2402.10351v3>] Quantum Automating TC^0 -Frege Is LWE-Hard

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def lex_dp11(F):
    def dp11(F, assignment):
        if not F:
            return True
```

```

unit_clauses = [c for c in F if len(c) == 1]
if unit_clauses:
    v = unit_clauses[0][0]
    if v < 0 and -v in assignment and assignment[-v]:
        return False
    elif v > 0 and v not in assignment:
        assignment[v] = True
    else:
        assignment[v] = False
pure_literals = [v for v in range(1, max(F) + 1) if all(v in c or -v in c for c in F)]
if pure_literals:
    v = pure_literals[0]
    if v not in assignment:
        assignment[v] = True
    else:
        assignment[v] = False
for literal in range(1, max(F) + 1):
    if literal not in assignment and -literal not in assignment:
        return dpll(F, {**assignment, literal: True}) or dpll(F, {**assignment, literal: False})
return False

return dpll(F, {})

def grundy_value(F, memo):
    F = [tuple(sorted(c)) for c in F]
    if tuple(sorted(F)) in memo:
        return memo[tuple(sorted(F))]
    if not F:
        return 0
    moves = set()
    for clause in F:
        for literal in clause:
            new_F = [c for c in F if literal not in c and -literal not in c]
            new_F = [tuple(sorted(c)) for c in new_F]
            moves.add(grundy_value(new_F, memo))
    mex = 0
    while mex in moves:
        mex += 1
    memo[tuple(sorted(F))] = mex
    return mex

def generate_cnf(n, alpha):
    F = []
    variables = set(range(1, n + 1))
    for _ in range(int(alpha * n * (n - 1) / 2)):
        v1, v2 = random.sample(variables, 2)
        literals = [v1, -v1, v2, -v2]
        random.shuffle(literals)
        F.append(tuple(literals))
    return F

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 10, 12]
    alpha_values = [4.5, 5.0, 5.5]
    memo = defaultdict(int)

```

```

instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for alpha in alpha_values:
        F = generate_cnf(n, alpha)
        if lex_dp11(F):
            continue
        G_F = grundy_value(F, memo)
        L_T = lex_dp11([tuple(sorted(c)) for c in F], {})
        instances_tested += 1
        if not (math.floor(math.log2(1 + G_F)) <= math.ceil(math.log2(L_T))):
            conjecture_holds = False
            counterexample = f"n={n}, alpha={alpha}, G(F)={G_F}, L_T={L_T}"
            break

    return {
        "metric_name": "log2_GF_vs_log2_LT",
        "metric_value": math.log2(1 + grundy_value(generate_cnf(10, 5.0), memo)) if instances_tested > 0 else None,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["instances_tested"] > 0) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["instances_tested"] > 0) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad3b71d3.py", line 107, in <module>

```

result = run_trial(seed)
^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad3b71d3.py", line 83, in run_trial
    F = generate_cnf(n, alpha)
    ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ad3b71d3.py", line 66, in generate_cnf
    v1, v2 = random.sample(variables, 2)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence.  For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `generate_cnf` (`random.sample` called on a non-sequence) before any instances were evaluated, so neither the violation count nor the Pearson correlation was measured. | next: Fix `generate_cnf` to pass a list (e.g., `random.sample(list(variables), 2)`) and rerun the pre-registered 300-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	144013
2	preregistration	claude_max	opus	0	0	6560
3	novelty	claude_max	opus	0	0	2960
4	novelty	claude_max	opus	0	0	8273
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18039
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12862
7	judge	claude_max	opus	0	0	5344

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 198052 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/501b089a21b9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/501b089a21b9.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/501b089a21b9.tar.gz` (if generated)